

Implement Client Script & UI Policy (Incident)

Phase 4 - Project Planning Phase

The **Project Planning Phase** turns the approved design into an actionable implementation plan for the ServiceNow Incident form. The focus is on **tasks, sequence, ownership, dependencies, testing, and deliverables**.

1. Project Objective

Implement and validate **Client Scripts and UI Policies** on the Incident table to provide the required dynamic form behavior while minimizing unnecessary scripting.

2. Implementation Plan

Phase	Task	Deliverable
1	Review Incident requirements	Approved requirement list
2	Confirm affected fields	Field mapping
3	Review existing UI Policies/Scripts	Conflict/dependency list
4	Configure UI Policies	Working UI behavior
5	Develop Client Scripts	Working dynamic behavior
6	Unit test	Test results
7	Integration/regression test	Validation results
8	User acceptance testing	UAT approval
9	Prepare deployment	Update Set/package
10	Deploy and verify	Production validation

3. Work Breakdown Structure

Task 1 — Requirement Confirmation

Confirm:

- Incident states involved.
- Fields affected.
- Visibility requirements.
- Mandatory requirements.
- Read-only requirements.
- Dynamic behavior.
- Validation requirements.
- User/role-specific behavior.
- Target interfaces, such as standard UI and/or Workspace.

Output: Finalized requirement matrix.

Task 2 — Instance Assessment

Before development, inspect existing Incident configuration:

- Existing UI Policies.
- Existing Client Scripts.
- Dictionary settings.
- Business Rules affecting the same fields.
- Data policies.
- Existing customizations.
- Potential naming conflicts.

Output: Dependency/conflict assessment.

Task 3 — UI Policy Development

Implement simple form behavior first.

Example:

UI Policy

↓

Condition: State = Resolved

↓

Resolution code → Mandatory

Resolution notes → Mandatory

Test both:

State = Resolved

State ≠ Resolved

This ensures the policy behaves correctly in both directions.

Task 4 — Client Script Development

Implement only the logic that requires scripting.

Planned scripts:

onLoad

↓

Initialize existing Incident state

onChange
↓
React to Category / dependent-field changes

onSubmit
↓
Perform client-side validation

Scripts should be developed independently and tested before combining their behavior.

4. Development Sequence

The recommended sequence is:

Requirements
↓
Existing configuration review
↓
UI Policies
↓
onLoad Client Script
↓
onChange Client Script
↓
onSubmit Client Script
↓
Unit Testing
↓
Integration Testing
↓
UAT
↓
Deployment

This sequence prioritizes declarative configuration before custom JavaScript.

5. Test Planning

Create test cases before implementation is considered complete.

Test Case	Action	Expected Result
TC01	Create new Incident	Form loads correctly
TC02	Open existing Incident	Existing values are handled correctly
TC03	Set State to Resolved	Resolution fields behave as designed
TC04	Change State away from Resolved	Resolution UI behavior reverses as designed
TC05	Change Category	Dependent behavior executes correctly

Test Case	Action	Expected Result
TC06	Enter invalid data	Appropriate validation appears
TC07	Submit valid Incident	Record saves successfully
TC08	Submit invalid Incident	Submission is blocked when required
TC09	Test with different roles	Role-dependent behavior is correct
TC10	Reload the form	Behavior remains consistent

6. Roles and Responsibilities

For a team implementation:

Role	Responsibility
Business Analyst	Confirm requirements
ServiceNow Developer	Configure UI Policies and Client Scripts
ServiceNow Admin	Review configuration and deployment
Tester	Execute functional/regression tests
Business User	Perform UAT
Release Manager	Coordinate deployment

For a small project, several of these roles may be performed by the same person.

7. Dependencies

Before development begins, confirm:

- Required Incident fields exist.
- Correct field names and choice values are known.
- Developers have appropriate permissions.
- Existing customizations have been identified.
- The target application/scope is confirmed.
- Update Set strategy is established.
- Test data is available.

8. Risk Planning

Risk	Impact	Mitigation
Existing Client Script conflicts	High	Review existing scripts first
UI Policy conflicts	High	Check policy order and conditions
Incorrect choice values	Medium	Verify actual dictionary/choice values
Client-side validation bypassed	High	Use server-side enforcement where required
Behavior differs in Workspace	Medium/High	Test the intended interfaces
Excessive JavaScript	Medium	Prefer UI Policies for simple behavior
Existing records behave	Medium	Include existing-record tests

Risk	Impact	Mitigation
differently		

9. Deployment Plan

Use the organization's normal ServiceNow deployment mechanism, such as an **Update Set** where appropriate.

Deployment sequence:

```

Development
↓
Unit Testing
↓
Test Instance
↓
Integration / Regression Testing
↓
UAT
↓
Deployment Approval
↓
Production
↓
Post-deployment Verification

```

10. Completion Criteria

The project can be considered complete when:

- All approved UI Policies are configured.
- Required Client Scripts are implemented.
- No unnecessary duplicate logic exists.
- Positive and negative scenarios pass.
- Existing Incident records behave correctly.
- Relevant interfaces have been tested.
- Regression testing is complete.
- UAT is approved.
- Deployment is completed successfully.
- Configuration/documentation is handed over.

Planning Deliverable

The final planning document should therefore establish:

PROJECT: Incident Client Script & UI Policy

Scope:

Incident form dynamic behavior

Configuration:
UI Policies

Customization:
Client Scripts

Testing:
Unit + Integration + Regression + UAT

Deployment:
Approved update/deployment process

Success Criteria:
Requirements implemented and all planned
test cases successfully validated.

This provides the bridge from the **Project Design Phase** into the
Development/Implementation Phase.